

Full demo script — word for word

Everything below is meant to be **read aloud as written**. It is timed to the prototype, so if you read at a normal pace the words land while the matching thing is on screen.

Total: **3 minutes 05 seconds**. Two segments, no gap between them.

Legend:

- **[ON SCREEN]** — what the audience is looking at. Do not read this aloud.
- **[DO]** — an action you take. Do not read this aloud.
- Everything else is spoken.

Pre-flight — do this before you press record

Two terminal tabs, both inside the project folder.

Tab 1:

```
cd /Users/om/Downloads/mycroft-ai && ./venv/bin/python -m streamlit run app/streamlit_app.py
```

Wait for the browser to open at `http://localhost:8501`, then leave it alone.

Tab 2 (`Cmd + T` for a new tab):

```
cd /Users/om/Downloads/mycroft-ai
```

Do **not** run the demo yet. Type this command and leave the cursor sitting there, unexecuted:

```
./venv/bin/python scripts/record_demo.py
```

Then: terminal font at 16–18 pt, notifications off, Tab 2 filling the screen.

Press record. Wait two seconds of silence. Begin.

SEGMENT A — the terminal • 0:00 – 1:20

0:00

[DO] Press Return to start the demo.

Enterprises don't run one AI system. They run several at once — a support chatbot, an internal copilot, a tool that helps decide something about a person. Each one carries a different risk, and a different tolerance for being slow. This is Mycroft.ai, and it governs all of them from one place.

0:12 — Section 1 appears

[ON SCREEN] *A FAILURE REACHES A CUSTOMER*

Here's a real interaction. A customer asks when their refund will arrive. The assistant answers: five working days.

Now look at the line underneath. That's the policy document the assistant was given. It says seven to ten.

The answer is fluent, it's confident, and it's wrong. Nothing crashed. Nothing threw an error. This is what AI failure actually looks like.

0:30 — Section 2 appears

[ON SCREEN] *TWO LANES, ONE ACTION*

Two lanes check every interaction.

The Fast Lane is deterministic and runs inline, on a hundred percent of traffic. A third of a millisecond. That's the lane that's allowed to block a response, because it's cheap and it's predictable.

The Slow Lane is the semantic one. It runs asynchronously, so it never sits between the user and their answer.

And here's the finding. High severity, hallucination, confidence nought-point-nine — and crucially, it's pinned to evidence. What the response said, and the exact sentence in the policy that contradicts it. A reviewer never has to take our word for it.

Action: block.

0:52 — Section 3 appears

[ON SCREEN] *THE SAME TRACE, THREE JURISDICTIONS*

Now a different interaction, run three times.

Same trace. Same code. Three policy packs.

India sends it to human review. The European Union blocks it. The United States reviews it.

Nobody wrote an if-statement for that. The EU pack raises the high-severity action from review to block, and it cites Article 22 for doing it. Regulation is configuration here, not application logic.

1:10 — Section 4 appears

[ON SCREEN] *FORTY FINDINGS BECOME ONE PRICED DEFECT*

One bad answer is a finding. Nobody can act on four thousand of those.

So we cluster them. Sixteen traces, one defect, one root cause — the system prompt never told the assistant to quote the policy it was given.

And it has a price. About three and a half lakh a month.

That number isn't invented. It's the measured rate, times four assumptions that are printed directly underneath it. If you think a handoff costs two hundred rupees instead of ninety-five, you change one line and the number changes with you.

1:34 — Section 5 appears

[ON SCREEN] *A HUMAN DECIDES; THE CONTRACT CHANGES*

A human reviewer confirms it. Nothing moves without that step.

And confirming it writes a contract change. One flag flips — every answer now has to be backed by a real document. Revision one becomes revision two, and the contract gets a new hash.

1:50 — Section 6 appears

[ON SCREEN] *THE REGRESSION PROVES IT — AND REPORTS THE BILL*

Then we re-run the defect's own traces, plus every case a reviewer has since confirmed shows the same failure.

Failures reaching the customer: six, down to three. Cut in half.

And the cost — because there is always a cost. Eleven more false positives. Four more interactions landing in a human's queue, out of a hundred and fourteen.

We report both directions, every time. A control that only ever told you the good half wouldn't be worth trusting with the next one.

2:10 — Section 7 appears

[ON SCREEN] *AN AUDIT CHAIN, NOT A LOG*

And every step of that — the decision, the defect, the human's verdict, the contract change, the regression — is one record, each carrying the hash of the one before it.

That's not a log file. It's a chain. It verifies live, and nothing in it can be quietly edited after the fact.

SEGMENT B — the prototype • 1:20 – 3:05

[DO] Switch the recording to the browser window at `localhost:8501`.

2:25 — Live tab

[DO] Open the **Trace** dropdown. Pick any entry whose label contains `hallucination`.

Same engine, as a product. You pick an interaction —

[DO] Point at the action badge.

— and you get the decision, the two lane timings, and the evidence side by side. The response on one side, the governed source on the other.

2:38 — the policy-pack table

[DO] Scroll to the bottom of the Live tab.

And here's that same trace under all three jurisdictions, computed live as you watch. Nothing changed between those rows except which policy pack is loaded.

2:48 — the overlap case

[DO] In the **Trace** dropdown, pick an entry labelled `hallucination, privacy` — it has two labels.

The brief calls this case out specifically. A made-up detail about a person is a hallucination and a privacy leak at the same time.

We settle it with evidence rather than wording. If the personal data appears in the retrieved context, it's real data being disclosed. If it appears nowhere, the model invented it — which is worse, because there's no source to correct.

Both labels get recorded. One action gets taken: the strictest one any finding demands.

3:02 — Evaluation Contract tab

[DO] Click **Evaluation Contract**. Change `pii` from `standard` to `strict`.

This is the contract itself. Change one setting —

[DO] Point at the version hash, then the table below it.

— and the version hash changes, and the decisions move across a hundred and fourteen traces. And the table underneath shows what that change bought you and what it cost you. Both numbers, always.

3:18 — Defects tab

[DO] Click **Defects**. Select `DEF-EA14BE` from the dropdown.

The defect register. Sixteen occurrences, one owner, one root cause, and the pricing formula printed right there.

[DO] Scroll down and click **Accept, update the contract, run the regression**. Wait about five seconds.

A reviewer accepts it — and you get the contract diff you can actually read, and the regression underneath. Six escapes down to three, at the cost of eleven false positives.

That's the whole trade, stated in both directions, in about four seconds.

3:40 — close

[DO] Stop scrolling. Let the numbers sit on screen for two seconds.

Anyone can flag a PII leak. Almost nobody turns forty of them into one owned, priced defect, carries a human's decision into a versioned contract, and then proves with a regression run that the fix actually worked — and tells you what it cost.

Every number you just saw was computed live, on a laptop, with no API key and no internet.

[DO] Stop recording.

If you need a shorter cut

Drop these and you land at about **1 minute 50**:

- Section 3 (three jurisdictions) — keep the app's version of it instead
- Section 7 (audit chain)
- The Evaluation Contract tab

Never drop **Section 6** or the Defects tab. The regression with its cost reported is the strongest thing in the whole demo.

Words that are easy to fumble

WRITTEN	SAY
₹3,54,787	"about three and a half lakh"
0.90	"nought point nine"
p95	"ninety-fifth percentile"
DEF-EA14BE	"the refund defect"
customer_support@r2-5ae442dedc	"revision two"

Two things not to claim

The script never says the system knows a statement is **false**. It says unsupported by, or contradicting, the governed documents. Keep that.

The script never says the system **detects bias**. It detects divergence and sends it to a human. If someone asks, that's the answer.